

■ Cours Express — Catégorie 2

■ Applied AI, Agentic Workflows & Enterprise Chatbots (Mistral FDE - PRIORITÉ #1)

1. ■ Gestion d'État de Conversation & Slicing

A. Effacement du System Prompt lors du Slicing de Mémoire

Problème : Faire `messages = messages[-10:]` supprime le `system_prompt` situé à l'index 0.

Impact Scalabilité / Structure : L'agent perd ses consignes de comportement, ses guardrails et son formatage.

■ Code Buggué	■ Correction Propre
<pre>messages = get_history() messages = messages[-10:] # ■ Efface system prompt en index 0 !</pre>	<pre>system_msg = messages[0] history = messages[1:][-10:] messages = [system_msg] + history # ■ Preserve system prompt</pre>

B. Fuite de Données Inter-Locataires (Cross-Tenant Data Leakage)

Problème : Relever des documents RAG sans appliquer de filtre `metadata tenant_id` strict.

Impact Sécurité : Faille critique multi-tenant. Les données confidentielles du Client A fuient vers le Client B.

■ Code Buggué	■ Correction Propre
<pre>results = vector_store.search(query, k=5) # ■ Pas de filtre tenant !</pre>	<pre>results = vector_store.search(query, filter={'tenant_id': current_user.tenant_id}, k=5) # ■ Contexte isolé</pre>

2. ■■ Tool Calling, Streaming & Robustesse API

A. Exécution d'Outils avec Arguments Non Validés

Problème : Invoquer `tool(**json.loads(args))` directement.

Impact Scalabilité : Injections de code ou crashes 500 récurrents si le LLM génère un format inattendu sous charge.

■ Code Buggué	■ Correction Propre
<pre>args = json.loads(tool_call.function.arguments) result = execute_sql(**args) # ■ Exécution brute dangereuse !</pre>	<pre>class SQLArgs(BaseModel): query: str validated = SQLArgs.model_validate_json(args_str) result = execute_sql(validated.query) # ■ Validation Pydantic</pre>

B. Hardcoding du Modèle Mistral Large pour Toutes les Tâches

Problème : Utiliser `mistral-large-latest` pour des étapes simples (extraction, routage).

Impact Coût & Débit : Coûts API 10x plus élevés et débit inutilement limité sur des tâches triviales.

■ Code Buggué	■ Correction Propre
<pre>def route_user_query(text: str): # ■ Over-provisioning inutile ! return client.chat.complete(model='mistral-large-latest', ...)</pre>	<pre>def route_user_query(text: str): # ■ Router vers mistral-small pour le routage rapide return client.chat.complete(model='mistral-small-latest', ...)</pre>

■ Réflexes Oraux pour l'Entretien Mistral (Jour J)

1. **'Attention au slicing de mémoire'** → Toujours préserver `messages[0]` (system prompt) avant de tronquer les messages.
2. **'Isolation Multi-Tenant obligatoire'** → Injecter systématiquement `filter={'tenant_id': ...}` dans les Vector Stores.
3. **'Validation Pydantic du Function Calling'** → Ne jamais faire `tool(**json.loads())` sans schéma de validation.
4. **'Router de Modèles (Mistral Small vs Large)'** → Utiliser Mistral Small pour les tâches simples afin de réduire latence et coût.